

GIT BUILD SERIES

# WORKBOOK 4

## Agentic Systems — Step by Step, Formulas & Frameworks

Six battle-tested AI agent systems extracted from live operations. Each one documented in CLAW structure with step-by-step execution formulas, revenue mechanics, and operational playbooks. From cold outreach to grant arbitrage — these are the machines that print money.

CLAW FRAMEWORK

AGENT SYSTEMS

4 MOOLAH

CONTENTS

# Table of Contents

THE DNA — AGENT ARCHITECTURE

|                                                         |     |
|---------------------------------------------------------|-----|
| Module 1: The CLAW Framework — Your Agent Design System | p.3 |
| Module 2: The Cold Assassin — Lead Gen System           | p.6 |
| Module 3: The Meeting-to-Workflow Agent                 | p.9 |

GIT MONEY — REVENUE AGENTS

|                                                                |      |
|----------------------------------------------------------------|------|
| Module 4: The Complaint Harvester — Digital Product Engine     | p.12 |
| Module 5: The Vibe-Coded Game Builder — IP Production Pipeline | p.15 |
| Module 6: The Merch Drop Pipeline — Apparel Rapid Mockup       | p.18 |

THE MATRIX — SYSTEMS & SCALE

|                                                                  |      |
|------------------------------------------------------------------|------|
| Module 7: The Grant Evidence Engine — Board Minutes to Arbitrage | p.21 |
| Module 8: Cross-Cutting Patterns & Operational Playbook          | p.24 |
| Cheat Sheet                                                      | p.27 |
| Final Assessment (20 Questions)                                  | p.29 |

MODULE 1

## The CLAW Framework — Your Agent Design System

Every agent in this workbook follows the same 4-stage architecture: CLAW (Capture, Logic, Action, Workflow). This isn't theory — it's the operating system that Red Pillar Workflows uses to turn live war stories into documented, repeatable, sellable systems. Most people demonstrate brilliance in a meeting and lose it forever. CLAW captures it, structures it, and makes it runnable by anyone. This module gives you the meta-framework before we dive into each specific agent. By the end of this module, you will be able to map any business process onto the CLAW structure and identify exactly where automation should be applied for maximum impact. Every system that follows in this workbook is built on these four pillars, so internalize them now before we get into the specifics of each agent.

## C — Capture (Data Ingestion)

The first stage defines WHAT data the agent consumes and from WHERE. Every agent starts with a defined input source — public data scrapes, Zoom AI notes, negative reviews, concept briefs, visual research, or meeting recordings. The key discipline: your capture source must be automated or semi-automated, not manual. If you're copy-pasting by hand, you don't have an agent — you have a chore. The capture stage determines the quality of everything downstream, so invest time in building reliable, repeatable data ingestion pipelines that run without your direct involvement. Think of it as the foundation of a house: if the data coming in is noisy, inconsistent, or late, nothing downstream can compensate for it. The best capture systems run on schedules or triggers, pulling fresh data at regular intervals without anyone pressing a button.

DNA

## L — Logic (Processing Engine)

The second stage is the brain. Raw captured data enters the logic layer, where it gets filtered, analyzed, diagnosed, or transformed. This is where the agent's specific intelligence lives — the diagnosis engine in Cold Assassin, the two-stage analysis in Complaint Harvester, the legal clearance analysis in Game Builder. The logic layer turns noise into signal. A poorly designed logic layer produces garbage outputs no matter how good the capture is. The best logic layers are narrow and specific: they do one transformation extremely well rather than trying to be general-purpose. Think of it like a surgical instrument, not a Swiss Army knife. When the logic layer is too broad, the outputs become generic and useless. When it's laser-focused on one type of transformation, the results are sharp enough to act on immediately.

DNA

## A — Action (Output Generation)

The third stage produces a concrete artifact — not just analysis, but something someone can act on immediately. A one-pager pre-call asset. A named workflow. A ranked list of digital product ideas. A game spec document. A tech pack ready for factory dispatch. The action stage is where intelligence becomes deliverable. If your agent only produces insights but no artifacts, it's an analyst, not an operator. The action stage must produce output that is specific enough for a stranger to execute without additional context. The test is simple: hand the output to someone who was never in the room. If they can act on it without asking questions, the action stage is working. If they need clarification, the artifact isn't specific enough.

DNA

## W — Workflow (Orchestration & Recursion)

The fourth stage connects everything into a repeatable, triggered system with feedback loops. The workflow defines: What triggers the agent? What happens to the output? How does the system learn from its own performance? The best agents are recursive — their outputs become their own optimization inputs. This stage turns a one-time task into a standing system. Without it, you have a cool demonstration, not a business process. The workflow stage is what separates a project from a product, and an employee from an operator. Think of it as the circulatory system that keeps the agent alive: triggers pump data in, outputs flow to the right places, and feedback loops bring performance data back to improve the logic layer. Without workflow, an agent is a one-off. With it, it's a machine.

DNA

## The Self-Healing Sprint Cycle

Agent 1 (Cold Assassin) demonstrated a pattern every agent should follow: the system flags its own health defects and generates a fix plan from its own outputs, then optimizes twice (Codex plan, Fable 5 refinement) before redeploying. This makes agents anti-fragile — they get stronger every time they run. The cycle works because the agent treats its own failures as data, not as losses. Each failed batch becomes the training data for the next successful batch. This is the pattern that turns a static tool into a living, improving system. The self-healing cycle is what separates a system that degrades over time from one that compounds in capability. After 10 self-healing cycles, the agent is fundamentally better than it was on day one — not because someone improved it, but because it improved itself.

DNA

## The Canary Test Protocol

Before any batch operation, test one output in isolation. Check for "AI slop" — robotic phrasing, generic filler, or tone-deaf messaging. Only after the canary passes does the batch fire. This single discipline prevents reputation damage at scale. The canary test takes five minutes and can save you from sending 500 emails that sound like they were written by a robot. It is the cheapest insurance policy in agentic operations, and every agent in this workbook uses it. The canary test isn't about perfection — it's about catching the obvious failures that would destroy credibility. A single AI-sounding email to the wrong prospect can burn a relationship permanently. The canary prevents that by catching problems before they multiply.

DNA

## \$ GIT MONEY

CLAW is how you turn a 200/hour consultant into a 0/hour system. Every agent in this workbook replaces expensive human labor with structured automation. The Cold Assassin replaces a lead gen team. The Complaint Harvester replaces a product research department. The Meeting-to-Workflow Agent replaces a chief of staff. That's not efficiency — that's leverage. And leverage is what separates operators from employees. When you can run the same process at zero marginal cost that used to require a full-time hire, you've fundamentally changed your economics. The CLAW framework isn't just a design tool — it's a wealth-creation tool, because every agent you build using this structure eliminates a line item from your payroll while producing equal or better output.

## CLAW-Map Your Current Operation

- 1. Identify one process in your business:** Pick a repeatable task that takes real time every week — lead generation, client onboarding, content creation, meeting follow-up, or product research. Write down exactly what it is, who currently performs it, and approximately how many hours per week it consumes. Be honest about the time cost, including the invisible hours (thinking, planning, context-switching) that don't show up on any timesheet.
- 2. Document the current C-L-A-W stages:** For your chosen process, write down what data is currently captured (and how it's captured), how it's processed or analyzed, what output is produced, and whether it runs on a schedule or triggers automatically. Be brutally honest — most processes have zero automation at the Capture and Workflow stages. Most "systems" are really just people doing things from memory.
- 3. Identify which stage is weakest:** Almost every manual process breaks at Capture (no automated data source) or Workflow (no scheduling, no feedback loop). Identify your weakest stage and explain why it's the bottleneck. Is it because no tool exists, because you haven't set it up, or because you didn't know it was possible? The diagnosis matters because it tells you exactly what to fix first.
- 4. Design one automation for the Capture stage:** Write a specific plan for automating the data ingestion. Could you use a webhook, an API, a scheduled scraper, or a Zapier integration? What tool would you use, what would the automated input look like, and how often would it run? Write this as if you're giving instructions to someone who will implement it tomorrow.
- 5. Write a canary test for the Action stage:** Before you would scale this process, design a single-output test. What exactly would you check? What would constitute a "pass" versus a "fail"? Write 3-5 specific criteria. This canary test becomes your quality gate — the thing that stands between you and a batch of bad outputs.

Write your CLAW map here:

---

---

---

---

---

## QUIZ

**1. What does the "C" in CLAW stand for?**

A) Create B) Capture C) Configure D) Compute

**2. What is the key discipline for the Capture stage?**

A) Manual data entry for accuracy B) The capture source must be automated or semi-automated, not manual C) Only use paid data sources D) Capture everything possible regardless of relevance

**3. What makes the best agents "recursive" according to the CLAW framework?**

A) They run on a loop forever B) Their outputs become their own optimization inputs C) They require human intervention each cycle D) They use recursive programming languages

**4. What is the purpose of the canary test protocol?**

A) To test if the AI model is functioning B) To test one output in isolation for AI slop before firing a batch C) To measure email response rates D) To validate data source quality

**5. What distinguishes an "operator" from an "employee" according to the CLAW framework?**

A) Operators work longer hours B) Operators build systems with leverage at the Workflow stage C) Operators use more software tools D) Operators have better job titles

## MODULE 2

# The Cold Assassin — Lead Gen System

This is Hitsuyo Aku's personal lead generation system, demonstrated live during Class 2. It generates hyper-contextualized cold outreach that proves deep research was done without faking familiarity. The system uses public data to diagnose gaps between what prospects claim and what their assets actually deliver, then produces surgical one-pagers that prospects can act on immediately. This is a real, sellable Triple AI Agency product — and it's currently running entirely inside one person's build. The Cold Assassin is the first revenue agent we're covering because it demonstrates every CLAW principle in action, from automated data capture to self-healing optimization loops. This module will teach you the complete system, step by step, so you can build your own version or sell it as a service.

## Public Data Scrape + ICP Filtration (Capture)

The agent pulls public data on qualified prospects — their website, reviews, LinkedIn profiles, and public service pages. Every prospect passes through budget-qualification gates: "Can they afford a \$10K service without haggling?" This isn't about casting a wide net. It's about building a narrow funnel that only lets through prospects who can actually pay. The filtration discipline is what separates a lead gen system from a spam machine. Every prospect that enters the pipeline has already been pre-qualified on budget before a single word of outreach is written. The capture runs on a schedule, pulling fresh data on a regular cadence so the pipeline never runs dry. The data is structured and stored for the logic layer to process.

DNA

## The Diagnosis Engine (Logic)

This is the brain of Cold Assassin. It identifies the specific gap between what the prospect claims publicly and what their public assets actually deliver. Not personalization — hyper-contextualization. The agent proves research was done without faking familiarity. The output isn't "Hi [Name], I noticed your company..." — it's a precise diagnosis of a problem the prospect already knows they have but hasn't fixed. This is the intelligence layer that turns generic outreach into something that makes the recipient stop and think, "How did they know that?" The diagnosis engine is what makes the Cold Assassin lethal: it targets problems the prospect is already feeling, which means the outreach doesn't need to create urgency — the urgency already exists.

DNA

## The Locked Outreach Structure (Action)

Every outreach artifact follows a locked six-part structure: Observation, Boundary, Why It Matters, Artifact, Next Move, and CTA. The Artifact is the key differentiator — a "one-pager" pre-call asset the prospect can act on immediately. This isn't a pitch. It's proof of competence delivered before the first conversation. The locked structure ensures consistency across every single outreach piece while allowing the content to be fully customized per prospect. It eliminates the blank-page problem and ensures every email hits the same psychological triggers in the same order. The six-part structure was refined through iteration and is now locked — meaning it doesn't change, only the content within each section changes per prospect.

DNA

## Canary-Batch Fire Protocol (Workflow)

The system fires in scheduled batches (noon and 3pm). But before any batch sends, one email per batch is canary-tested for AI slop. If the canary fails, the whole batch stops. After firing, every non-reply is logged as a learning event (message refinement), not a loss. Reply rate is the leading indicator of revenue. This workflow stage is what makes Cold Assassin a system rather than a tactic. The scheduled batching, the canary gate, and the feedback loop together create a machine that improves itself with every cycle. Non-replies are not treated as failures — they are treated as data points that improve the next batch's messaging. This reframing is critical: most people give up after low response rates. Cold Assassin gets smarter from them.

DNA



## The Self-Healing Optimization Loop

The system flags its own health defects and generates a fix plan from its own outputs. The optimization runs through two stages: Codex (planning) then Fable 5 (refinement) then redeploy. Each cycle makes the next batch smarter. This is the pattern that turns a good system into an unfair advantage over time. The self-healing loop means the agent doesn't just maintain performance — it compounds it. Every failure is data that makes the next attempt more precise. After 20 optimization cycles, the Cold Assassin is a fundamentally different machine than it was on day one. This compounding intelligence is what makes it so dangerous as a competitive weapon — the system literally learns from its own mistakes.

4 MOOLAH

## \$ GIT MONEY

The Cold Assassin replaces a 5,000/month lead generation team. At a 10K service price point, you need only 3-5 new clients per month to generate 30 — 50K in revenue. If the system achieves even a 25,000/month whether they perform or not. The Cold Assassin costs a few dollars in API calls and produces outputs that improve over time — something no human team can match.

## Build Your First Cold Assassin Diagnosis

1. **Pick a real prospect:** Choose a business you'd genuinely want to work with. Go to their website and their public review profiles (Google, Trustpilot, Yelp). Write down who they are, what they claim to do, and what you find in their public reviews. Spend at least 15 minutes on this — the quality of your diagnosis depends entirely on the quality of your research.
2. **Scrape their public assets:** Read their website copy, their social media profiles, and at least 10 customer reviews. Document the gap: what do they say they deliver versus what do customers actually experience? Be specific — find at least one concrete, undeniable discrepancy that the prospect already knows about but hasn't fixed.
3. **Identify the diagnosis:** Write a one-sentence diagnosis of the prospect's core problem based on the gap you found. This is not a pitch — it's an observation that proves you did real research. The prospect should read it and immediately think, "That is exactly our problem." If it sounds like marketing, it's not a diagnosis.
4. **Write a 6-part outreach following the locked structure:** Using the Observation, Boundary, Why It Matters, Artifact, Next Move, and CTA framework, draft a complete outreach email. Include a specific one-pager artifact description that the prospect could actually use. The email should be under 200 words.
5. **Design a canary test:** Before you would send this email, what specific things would you check? Write 3-5 criteria that would determine whether this email passes or fails the canary test. What would "AI slop" look like in this specific context? Write your canary checklist.

Write your diagnosis and outreach here:

---

---

---

---

---

## QUIZ

### 1. What is the key difference between personalization and hyper-contextualization?

A) Hyper-contextualization uses more data points B) Hyper-contextualization proves research was done without faking familiarity by diagnosing real gaps C) Personalization is automated, hyper-contextualization is manual D) There is no difference — they are the same thing

### 2. What is the purpose of the budget-qualification gate in the Capture stage?

A) To estimate the prospect's annual revenue B) To filter for prospects who can afford the service without haggling C) To determine the price of the outreach email D) To qualify for CRM software credits

### 3. How many parts are in the Locked Outreach Structure?

A) 3 B) 4 C) 5 D) 6

### 4. What happens when a canary test fails in the Cold Assassin system?

A) The canary email is revised and retested B) The whole batch stops and does not fire C) The batch fires anyway with the flawed email removed D) The system switches to a different AI model automatically

### 5. What are non-replies logged as in the Cold Assassin system?

A) Dead leads to be permanently deleted B) Failures that indicate the system is broken C) Learning events for message refinement, not losses D) Spam complaints requiring removal

## MODULE 3

# The Meeting-to-Workflow Agent

This is Red Pillar's own meta-workflow — the agent that generates other agents. It takes raw meeting transcripts and automatically converts them into structured, named, CLAW-formatted workflows. During Class 2, Phillip demonstrated this live: Zoom AI notes were pasted into Claude with a single instruction, and out came fully structured workflows including "Sponsorship Activation Loop" and "Showcase & Contributor Invite Pipeline." This is the most important agent in the entire workbook because it's recursive — meetings produce the infrastructure that runs future meetings. If you only build one agent from this workbook, this should be the one. It is the system that generates all other systems, and it compounds in value with every meeting you run.

## Zoom AI Capture (Capture)

Zoom's native AI note-taking (enterprise tier) auto-generates meeting summaries and transcripts. Every Summer School session, every strategy call, every pod call becomes timestamped proof-of-work by default. The capture is zero-effort — turn on recording, and the system handles the rest. This is the ideal capture stage: zero marginal cost, zero marginal effort, and the data quality improves as Zoom's AI gets better over time. The key is making recording the default for every meeting, not an exception you have to remember. Once recording is the default, every single conversation becomes potential workflow fuel. The Zoom AI output becomes the raw material that the logic layer transforms into structured systems.

DNA

## CLAW Auto-Structuring (Logic)

The captured summary is processed through a single instruction: "Create workflows out of our notes." The logic engine breaks free-form discussion into structured CLAW workflows automatically. It identifies which parts of the conversation are Capture sources, which are Logic patterns, which are Action items, and which are Workflow triggers. The elegance of this system is that the instruction is identical every time — only the input changes. This is the ultimate reusable logic layer because the structure is fixed and only the content varies. The same prompt that turned a Summer School session into the Sponsorship Activation Loop can turn a client call into a Client Onboarding Pipeline. The logic is universal; the content is specific.

DNA

## Named Workflow Generation (Action)

The output isn't just notes — it's named, structured, reusable workflows. Each workflow has a title, a CLAW breakdown, and enough specificity that any team member could execute it without having been in the meeting. The demo produced "Sponsorship Activation Loop" and "Showcase & Contributor Invite Pipeline" in real time. The naming convention matters because unnamed processes can't be referenced, delegated, or improved. A named workflow is a documented asset. An unnamed process is just a habit. The action stage ensures that every meeting produces at least one reusable artifact that can be filed, searched, assigned, and executed independently of any individual.

DNA

## Recursive Infrastructure Building (Workflow)

This is the meta-pattern: meetings produce workflows, workflows run future operations, future operations generate more meetings, more meetings produce more workflows. Each cycle adds to the workflow library. After 10 sessions, you have 30-50 documented systems. After a quarter, you have an operational playbook that no competitor can replicate because it was built from your specific, lived experience. The compounding effect is extraordinary — every meeting makes the organization smarter and more systematic at the same time. The workflow library becomes a proprietary asset that grows in value with every session you run. This is why the Meeting-to-Workflow Agent should be standing infrastructure: it turns every hour of meeting time into permanent organizational capital.

DNA

## \$ GIT MONEY

This agent replaces a \$150,000/year Chief of Staff. Think about it: someone whose entire job is to sit in meetings, take notes, follow up, and make sure decisions turn into actions. The Meeting-to-Workflow Agent does all of that automatically. Every strategy session, every client call, every team sync becomes a workflow factory. The ROI isn't just efficiency — it's the compounding value of a growing operational library that runs your business while you sleep. A Chief of Staff can only be in one meeting at a time. This agent processes every meeting simultaneously, perfectly, for the cost of an API call. Over the course of a year, the workflow library produced by this agent represents hundreds of thousands of dollars in consulting value, generated automatically from work you're already doing.

## Turn a Past Meeting Into a Workflow

1. **Pick a recent meeting:** Choose a meeting you had in the last week where decisions were made or action items were discussed. Write down the meeting topic, who was present, and what the main outcome was supposed to be. If you don't have a transcript, use your memory and any notes you took.
2. **Write down 3 decisions or action items:** From that meeting, extract three specific things that were decided or assigned. Be concrete — "follow up with the vendor" is too vague. "Email Mexico factory to confirm 200-unit strike-off timeline by Friday" is specific enough to execute. Vague items can't be systematized.
3. **CLAW-structure each one:** For each of the three decisions or action items, write out the full CLAW breakdown: Capture (what data triggers this workflow?), Logic (how is the data processed?), Action (what artifact is produced?), and Workflow (when does it repeat and what triggers the next cycle?).
4. **Name the resulting workflow:** Give each CLAW-structured item a specific, descriptive name. The name should be action-oriented and reference the outcome, not the meeting. Example: "Vendor Strike-Off Confirmation Loop" is better than "Tuesday Meeting Follow-Up." Names create identity, and identity creates accountability.
5. **Identify the automation opportunity:** Look at your three workflows and identify which one would have the highest impact if fully automated. Write down what tool or integration would make it run without human involvement. Estimate the hours per week this automation would save.

Write your workflows here:

---

---

---

---

---

## QUIZ

### 1. What single instruction drives the CLAW Auto-Structuring logic layer?

A) "Summarize this meeting in bullet points" B) "Create workflows out of our notes" C) "Find action items and deadlines" D) "List the key decisions made"

### 2. Why is the Meeting-to-Workflow Agent described as the "most important agent in the workbook"?

A) It generates the most direct revenue B) It's recursive — it generates the infrastructure for all other agents C) It's the easiest agent to build D) It was demonstrated first during Class 2

### 3. Approximately how many workflows can 10 sessions of meetings produce?

A) 5-10 B) 10-20 C) 30-50 D) 100+

### 4. What annual cost does this agent effectively replace?

A) 30,000 B) 75,000 C) 150,000 D) 300,000

### 5. Which of these was one of the workflows produced live during the Class 2 demo?

A) "Email Response Template Generator" B) "Sponsorship Activation Loop" C) "Client Onboarding Sequence" D) "Revenue Dashboard Setup"

## MODULE 4

# The Complaint Harvester — Digital Product Engine

Phillip demonstrated this live during Class 2 against the "GitHub repos and GitHub releases" niche. The agent scrapes negative reviews, frustrations, and complaints across the web, then runs a two-stage analysis to identify hidden demand gaps and generate 10 sellable digital product ideas mapped to each complaint pattern. The live test produced "Repo Translator Pack," "First-Run Fix Kit," and 8 others, ranked by fastest organic sales potential. This is the machine that turns complaints into cash. The Complaint Harvester is the first of our revenue-focused agents because it demonstrates the core principle of demand-driven product creation: you don't need to be creative, you just need to listen to what people are already complaining about and build the solution they're begging for.

## Multi-Platform Complaint Scraping (Capture)

The agent pulls negative reviews from Reddit, YouTube comments, Amazon reviews, Trustpilot, and social media comments across a specified niche. Complaints are segmented by user type — non-technical users, builders, maintainers — because each segment has different pain points and different willingness to pay for solutions. The multi-platform approach ensures coverage: what people complain about on Reddit is different from what they complain about in Amazon reviews, and both are different from YouTube comments. The segmentation by user type is what turns raw complaints into targetable product opportunities, because a 19 *prompt pack for beginners* and a 97 automation kit for professionals serve fundamentally different markets even within the same niche.

4 MOOLAH

## Two-Stage Demand Analysis (Logic)

Stage 1 identifies the number one desired outcome and the number one obstacle blocking it — the Hidden Demand Gap. Stage 2 generates 10 low-ticket digital product ideas (templates, swipe files, prompt packs, automation kits, mini-courses) mapped to each complaint pattern. Each idea gets: product name, core problem solved, hidden demand gap targeted, format, viral hook, why existing products fail, monetization potential, best platform, and buyer-intent score (1-10). This two-stage approach ensures that products aren't just ideas — they're scored, ranked, and mapped to real demand signals with specific data backing every recommendation. The buyer-intent score is particularly powerful because it measures not just interest but purchasing readiness.

4 MOOLAH

## Ranked Product List Output (Action)

The output isn't a brainstorm — it's a ranked, scored, sellable product list. Each idea has a buyer-intent score, a viral hook, and a specific format recommendation. The top 3 are flagged by fastest organic sales potential. Phillip's live test ranked items at positions #6, #2, and #1 as the top picks — meaning the best opportunities aren't always the most obvious. This scoring system is what turns a creative exercise into a data-driven product decision. You don't build what sounds cool — you build what the data says will sell fastest. The ranked output eliminates the "I don't know which product to build first" problem entirely. The data decides.

4 MOOLAH

## Niche-In, Products-Out Pipeline (Workflow)

The workflow is brutally simple: niche in, ranked and scored sellable product list out. Change the niche variable and the entire system reruns. This means the same agent can serve every Git Built cohort member, every Triple AI client, and every niche you want to explore — with zero structural changes. The pipeline is the ultimate example of a variable-input, fixed-structure agent. The CLAW architecture stays identical regardless of the niche, which means you build it once and run it infinitely across markets. The only thing that changes is the niche keyword. This makes the Complaint Harvester infinitely scalable at near-zero marginal cost per additional niche run.

DNA



## \$ GIT MONEY

The Complaint Harvester is a digital product printing press. Low-ticket products (templates, prompt packs, automation kits) sell for 19 —

*97 on Gumroad, Etsy, and Shopify. If each product generates just 50 sales/month at a 47% average, that's 2,350/month per product. Run 5 products simultaneously and you're at 11,750/month in digital product revenue — with zero inventory, zero shipping, and zero marginal cost. The only input is complaints. The internet has an infinite supply. Your only constraint is how many niches you're willing to run through the pipeline and how quickly you can produce the top-ranked products. This is the closest thing to a money printer that exists in the digital product space.*

### Harvest Complaints, Build Products

1. **Pick a niche:** Choose a specific niche where people actively complain online — software tools, fitness equipment, online courses, home office gear, or any category with active Reddit communities and Amazon reviews. Write down your chosen niche and why you chose it (active complaints = more data = better products).
2. **Scrape 10 complaints:** Go to Reddit, YouTube comments, and Amazon reviews for your niche. Copy 10 specific, detailed complaints. Note the user type (beginner, intermediate, expert) and the platform for each. Be specific — "this is confusing" is not actionable. "I can't figure out how to connect my GitHub repo to my deployment pipeline and the docs assume I already know Docker" is actionable.
3. **Identify the hidden demand gap:** From your 10 complaints, identify the single most common desired outcome and the single most common obstacle blocking it. Write this as a clear statement: "Users want [outcome] but can't because [obstacle]."
4. **Generate 3 product ideas with full scoring:** For each product idea, write: product name, format, price point, and buyer-intent score (1-10). Explain the viral hook and why existing products fail to address this specific gap. Score each honestly — don't inflate numbers.
5. **Pick the number one product to build first:** Based on your scoring, which product has the fastest path to first sale? Write a one-paragraph execution plan for how you'd create and launch it within 7 days using the CLAW framework.

Write your niche analysis here:

---

---

---

---

---

---

## QUIZ

**1. What is the "Hidden Demand Gap" identified in Stage 1 of the analysis?**

A) Products that have no reviews yet   B) The #1 desired outcome and the #1 obstacle blocking it   C) Gaps in a competitor's product lineup   D) Untapped geographic markets with no competition

**2. How many product ideas does Stage 2 of the analysis generate per complaint pattern?**

A) 3   B) 5   C) 10   D) 20

**3. Why are complaints segmented by user type in the Capture stage?**

A) To track which platform has the most complaints   B) Each segment has different pain points and different willingness to pay   C) To prioritize which complaints to address first   D) For legal compliance purposes

**4. What was the surprising finding from Phillip's live test about the top-ranked products?**

A) The most obvious idea always won   B) The best opportunities weren't always the most obvious (#6 ranked #1)   C) No products scored above 5 on buyer intent   D) Only one product was actually viable

**5. What makes the Complaint Harvester pipeline reusable across different niches?**

A) It requires custom code for each niche   B) The CLAW architecture stays identical — you only change the niche variable   C) It only works for software-related niches   D) Each niche needs a completely new AI model

## MODULE 5

# The Vibe-Coded Game Builder — IP Production Pipeline

Community member Andre Houser El demonstrated his pipeline for building "Thunderborn Ascension," a Kemetic/Atlantean 2D shooter, during Class 2. What makes this agent significant isn't the game itself — it's the production discipline: spec in a cheap model, build in a premium model, never the reverse. This token-economy principle is the single most important cost discipline in AI-assisted production, and Andre arrived at it independently through live building. This module teaches the full pipeline and the underlying economics. The Vibe-Coded Game Builder is a template not just for games, but for any creative production where the cost of iteration in a premium model would be prohibitive without a locked spec guiding the work.

## Concept Brief to Deep Research (Capture)

A concept brief is given to a deep-research-capable model (Claude/Cowork) specifying genre, IP-safety constraints ("recreate/remix, don't clone"), and platform flexibility ("whatever's easiest to start"). The research pass returns three critical outputs: legal clearance analysis (citing precedents like Atari v. Amusement World), engine recommendation (Godot — free, no royalties, commercial-ownership-friendly), and a build roadmap: identity, prototype, content, ship and protect. This capture stage does the heavy intellectual lifting upfront, ensuring the entire production is built on legally safe and technically sound foundations. The concept brief is the seed; the deep research is the soil preparation. Without this stage, you'd be building on uncertain legal ground with the wrong tools.

DNA

## Deliberate De-Derivatization (Logic)

During the build, Andre caught himself naming the project too close to an existing IP reference ("Atlantis Blueprint") and manually renamed it before locking in the spec. This is the logic stage at its finest — the system (and the operator) actively prevents IP contamination. Genre conventions are not protectable (court precedent), but specific names, characters, and story elements are. The discipline is: borrow the genre, never the brand. This principle applies far beyond games — any creative production that draws inspiration from existing work needs this de-derivatization gate to prevent legal exposure and to ensure the final product stands on its own rather than being a derivative work that could be challenged.

DNA

## Spec-in-Cheap, Build-in-Premium (Action)

The MD spec file is built in Claude/Cowork first — explicitly to avoid burning Fable 5 tokens on exploratory work. Only once the spec is locked does it transfer to Fable 5 for the final build. This sequencing is a token-economy discipline: use 0.01 / *query models for exploration* and 0.10 / *query models for production*. Over a 50-step build, this saves 60-80% on AI costs while maintaining output quality. The principle is universal: never use expensive tools for work that cheap tools can handle just as well. The spec document is the boundary between exploration and production, and it must be complete before any premium-model work begins.

4 MOOLAH

## Distribution via Discord + Higgsfield (Workflow)

The full pipeline: Idea, legal/technical deep research (cheap model), locked spec (MD file), premium build (Fable 5), playable prototype, Higgsfield for visual asset upgrade, tester recruitment via Discord link drop. Each stage has a defined tool, a defined input, and a defined output. No stage overlaps. No ambiguity. The distribution workflow ensures the product doesn't just get built — it gets tested, polished, and placed in front of real players through community channels. The Discord community serves as both testing ground and early adoption channel, creating a feedback loop that improves the product before public launch while building an audience simultaneously.

4 MOOLAH

The game industry is a  $200B + \text{market}$ . *Indie games built with AI — assisted pipelines can reach 10K-100K* in revenue on platforms like Steam and Itch.io. The Vibe-Coded Game Builder pipeline reduces development cost from  $50 - 500K$  (*traditional*) to under  $500$  (AI-assisted). That's a 100-1000x cost advantage. And the spec-in-cheap, build-in-premium discipline isn't just for games — it applies to every AI production: courses, apps, content libraries, software tools. Any creative project that requires iteration before production can benefit from this token-economy principle. The cost savings alone make this pipeline worth mastering, but the real value is the systematic approach that makes creative production repeatable and scalable.

### Plan Your First Vibe-Coded Production

1. **Pick a creative project:** It doesn't have to be a game. Choose any creative project that requires iteration before production: a course, an app, a content library, a music album, or a short film. Write down what it is and what the final deliverable looks like in concrete terms.
2. **Run a legal/technical research pass:** Using a cheap AI model, run a research pass on your project. What IP constraints exist? What tools or engines are recommended? What is the legal precedent for your type of creative work? Document at least three key findings from the research pass.
3. **Write a locked spec:** Create a structured spec document that defines: project name (de-derivatized), core concept, target platform, key features, art direction, and success criteria. This spec should be detailed enough that someone else could build from it without asking you questions.
4. **Plan the cheap-model vs. premium-model split:** List every stage of your production pipeline. For each stage, mark whether it should use a cheap model (exploration, iteration, planning) or a premium model (final output, production build). Estimate your cost savings from this split.
5. **Design your distribution pipeline:** Where will this product live? How will you get testers? How will you recruit early users and build an audience? Write a 5-step distribution plan from prototype to first sale, including specific platforms and channels.

Write your production plan here:

---

---

---

---

---

## QUIZ

### 1. What is the core token-economy principle demonstrated by the Vibe-Coded Game Builder?

A) Always use the cheapest model available for everything B) Spec in a cheap model, build in a premium model — never the reverse C) Use the premium model exclusively for best results D) Alternate between models randomly for variety

### 2. What IP safety constraint does Andre apply to his game concepts?

A) "Clone and reskin with minor changes" B) "Recreate/remix, don't clone" C) "Avoid all creative inspiration from existing work" D) "Only use public domain assets exclusively"

### 3. What game engine did the deep research pass recommend and why?

A) Unity — because it's the most popular B) Unreal — because of superior graphics C) Godot — free, no royalties, commercial-ownership-friendly D) GameMaker — because it's the easiest to learn

### 4. What is "Deliberate De-Derivatization" in practice?

A) Removing all genre conventions from the design B) Actively preventing IP contamination by renaming references and borrowing genre, not brand C) Making the product as generic as possible D) Copyrighting every single element of the design

### 5. Approximately how much does the AI-assisted pipeline reduce game development cost compared to traditional methods?

A) 10-20% reduction B) 50-60% reduction C) 100-1000x cost advantage (from 50 — 500K *tounder* 500) D) 100% reduction (completely free)

## MODULE 6

# The Merch Drop Pipeline — Apparel Rapid Mockup

Both Phillip and Andre Houser EI demonstrated parallel independent builds of the same pattern during Class 2: visual research, master prompt, systematic expansion, mockup generation, and tech pack staging. This is the merch drop pipeline — a system for going from concept to factory-ready tech pack in hours, not weeks. The pipeline already conforms to the locked apparel production rule: strike-off before bulk, no exceptions. What makes this agent remarkable is that two people arrived at the same system independently, which means it's not a personality-dependent trick — it's a structural law of apparel production with AI. This module teaches the complete pipeline from visual concept to pre-sale validation, so you can go from idea to revenue in days instead of months.

## Visual Research + Image Inputs (Capture)

Manual visual research feeds the system: IG search for trending streetwear silhouettes, Google image search for iconography and aesthetic references (USA250, Ralph Lauren aesthetic). These images become inputs to the model, grounding the output in real-world design language rather than abstract descriptions. The visual research stage is what separates AI apparel design that looks real from AI apparel design that looks obviously generated. Real reference images give the model a target aesthetic that abstract text prompts can never achieve. The capture stage produces a mood board of 10–15 reference images that define the visual DNA of the entire product line before a single design is generated.

DNA

## Master Prompt Expansion (Logic)

A single "master prompt" is locked first — designer persona, brand mandate, style references. Then it's systematically expanded into 15 prompts covering a full product line: tees, hoodies, sweatshirts, varsity jackets, shorts, jeans, sweatpants, stickers, bags. One master prompt becomes 15 production-ready designs. This is the formula: lock the DNA, then multiply. The master prompt ensures brand consistency across an entire product line while the expansion ensures coverage across every product category your customers expect. The logic layer is what prevents the common mistake of designing 15 disconnected pieces that look like they came from 15 different brands.

DNA

## Mockup to Tech Pack to Factory Dispatch (Action)

Mockups are generated, then a tech pack is built immediately upon design approval. The mindset: "I don't have to get ready, I'm already ready." The tech pack is staged for instant dispatch to factory (Mexico/Haiti/India/Pakistan) the moment demand is validated. The pipeline goes from design to production-ready in a single session. The tech pack is the critical artifact — it's the document that turns a design concept into a manufacturable product. Without it, you have art. With it, you have inventory. The tech pack includes: materials, dimensions, color codes, print specifications, stitching details, and packaging requirements — everything a factory needs to produce your product without a single follow-up question.

4 MOOLAH

## Pre-Sale Validation Loop (Workflow)

The full workflow: Design, mockup, landing page plus 100 — 200 *ad spend* test, pre — *sale validation*, tech pack dispatch, strike — *off sample*, bulk production. No inventory is purchased before demand is proven. The *ad spend* test is the 200, not \$20,000. This workflow is the financial discipline that turns fashion from a gamble into a calculated investment. The pre-sale loop means you never produce a single unit without confirmed demand. This is the canary test principle applied to physical products: validate with a small, cheap test before committing to a large, expensive production run.

4 MOOLAH

## \$ GIT MONEY

Apparel margins are 50-70% on direct-to-consumer. A hoodie that costs 18 to produce sells for 55-65. A tee that costs 6 sells for 25-35. If your pre-sale validation generates 200 pre-orders at 55 average, that's 11,000 in revenue against 3,600 in production cost—7,400 gross profit on a 200 ad spend test. The Merch Drop Pipeline turns fashion into a software business : test before you build, validate before you scale. The only risk is the ad spend, and even that teaches you something. 50K in startup capital.

### Design Your First Merch Drop

1. **Pick a visual aesthetic:** Choose a specific design direction with real visual references. Search Instagram and Google for 5-10 reference images that define your aesthetic. Write down the aesthetic name, the key visual elements (colors, typography, imagery style), and the target customer demographic.
2. **Create a master prompt:** Write a master prompt that includes: designer persona, brand mandate, color palette, typography direction, and style references. This prompt should be specific enough that someone else could read it and understand exactly what your brand looks like visually.
3. **Expand to 5 product variants:** Using your master prompt as the DNA, write 5 expansion prompts for different products (tee, hoodie, sweatshirt, hat, and one wildcard item). Each should reference the master prompt's aesthetic while adapting to the specific product form factor.
4. **Build a mockup concept:** Pick one product variant and describe the final mockup in complete detail. What does the front look like? The back? The sleeves? What materials? What specific colors (Pantone codes if possible)? Write a description detailed enough that a factory could produce it.
5. **Design your pre-sale validation test:** Write a plan for a \$200 ad spend test. What platform (Instagram, Facebook, TikTok)? What audience targeting? What's the landing page headline? What conversion rate would make you proceed to tech pack dispatch? Write your go/no-go criteria.

Write your merch drop plan here:

---

---

---

---

---

## QUIZ

### 1. What is the "Master Prompt Expansion" formula in the Merch Drop Pipeline?

A) Generate 15 completely random designs B) Lock the DNA with one master prompt, then systematically expand into 15 product variants C) Use the same exact prompt for every product D) Ask the AI to independently design a full collection

### 2. What is the locked rule of the apparel production pipeline?

A) Always produce in bulk first to get volume discounts B) Strike-off before bulk, no exceptions C) Never use overseas factories under any circumstances D) Design must be completely final before any mockup is created

### 3. How much is the recommended ad spend for pre-sale validation?

A) 50 B) 100-200 C) 500 – 1,000 D) 5,000+

### 4. What is a "tech pack" in the context of the Merch Drop Pipeline?

A) A shipping container for bulk orders B) A production-ready document that specifies materials, dimensions, colors, and every detail needed for manufacturing C) A software development kit for e-commerce D) A marketing brief for the product launch

### 5. What is the approximate gross profit on 200 pre-orders at 55 *averagewith* 18/unit production cost?

A) 3,600 B) 7,400 C) 11,000 D) 14,600

## MODULE 7

# The Grant Evidence Engine — Board Minutes to Arbitrage

This is the highest-leverage, lowest-effort agent in the entire workbook — and it was the most neglected during Class 2. The concept is simple: every recorded session becomes timestamped proof-of-work evidence. That evidence, combined with AI-customized grant templates, becomes a pipeline for government AI education grants, youth sports grants, and agriculture grants. Rob/Mahal has sourced templates. The only thing missing is an assigned owner. This module teaches the system and identifies exactly what needs to happen to activate it. The Grant Evidence Engine is the definition of found money — the work is already being done, the evidence already exists, and the only missing piece is someone to connect the dots and file the paperwork before the deadlines pass.



## Standing Recording as Evidence Capture (Capture)

Zoom's native recording/notes feature becomes a standing "board of directors meeting minutes" capture system. Every Summer School session, every strategy call, every community event becomes timestamped proof-of-work by default. Accumulated teaching history dating back to 2023 provides the organizational evidence that government grant programs require as qualifying criteria. The key insight: you're already doing the work. You're just not capturing it as evidence. The standing recording discipline costs nothing extra and transforms every operational hour into documentary evidence of organizational capacity. This is the lowest-effort capture stage of any agent in the workbook because the work is already happening — you're just changing how you document it.

DNA

## Grant Program Mapping (Logic)

Government AI grant listings require documented "organization" as qualifying evidence. The logic layer maps your existing proof-of-work streams to specific grant categories: AI education grants (Summer School), youth sports grants (Blue Pillar football/basketball programs), agriculture grants (farming/summer camp work). Each program has different deadlines, different requirements, and different award amounts. The logic layer cross-references your evidence against program criteria to identify the highest-probability matches. This mapping exercise is where the money is — it turns a pile of evidence into a targeted application strategy. Without the mapping, you have evidence but no direction. With it, you have a prioritized list of applications ranked by win probability.

4 MOOLAH

## Template Customization + Submission (Action)

Grant templates (sourced and itemized by Rob/Mahal) are run through AI for extension and customization per specific grant program. Each application is tailored to the program's evaluation criteria, required evidence attachments, and narrative format. The action stage produces submission-ready grant applications — not drafts, not outlines — finished documents ready to file. The AI customization ensures every application reads as though it were written specifically for that program, which is exactly what grant reviewers want to see. A generic application gets rejected immediately. A tailored application demonstrates that you understand the program's mission and that your organization is the right fit for the funding.

4 MOOLAH

## Parallel Submission Pipeline (Workflow)

The workflow submits across multiple qualifying programs in parallel. AI education, youth sports, and agriculture are each distinct grant categories that can be pursued simultaneously. The pipeline: Record everything by default, minutes become evidence, evidence plus itemized templates, customized applications, submit across multiple programs. The scale potential: \$20-30M cited across qualifying programs. The parallel submission strategy maximizes the probability of winning at least one award while the total effort remains nearly identical to submitting a single application. Each additional application costs minimal incremental effort because the base evidence and templates are shared across all submissions.

4 MOOLAH

## \$ GIT MONEY

Grants are free money — not loans, not equity, not revenue-share. They are non-dilutive capital that does not require repayment. The Grant Evidence Engine turns your existing operations into a grant application factory. Every session you run, every student you teach, every community event you host becomes evidence that strengthens your applications. The marginal effort is near zero. The potential award amounts are life-changing. The only thing standing between you and this money is assigning someone to own the process. This is the lowest-hanging fruit in the entire workbook because the input (recorded sessions) already exists and the output (cash) requires no repayment, no equity dilution, and no ongoing obligation.

## Map Your Grant Evidence

1. **Inventory your existing proof-of-work:** List every recorded session, workshop, community event, or teaching hour you have documentation for. Go back as far as you can — 2023 if possible. Write down: what was the event, when did it happen, approximately how many people attended, and do you have timestamps or recordings?
2. **Identify 3 grant categories you qualify for:** Based on your proof-of-work inventory, identify at least three government grant categories where your documented evidence would meet qualifying criteria. For each one, write the category name and specifically why your evidence qualifies you.
3. **List the evidence requirements for each:** For each grant category, research and list the specific evidence requirements: organizational history, participant counts, timestamps, financial documentation, partner letters. Note which requirements you already meet and which you still need to build.
4. **Write a one-paragraph narrative:** Connect your existing work to one specific grant program. This paragraph should explain: who you are as an organization, what you've done (with specific evidence references), what you will do with the grant money, and why your organization is uniquely positioned to deliver results.
5. **Set a deadline for your first application:** Grants have fixed deadlines — they don't wait. Pick the grant program with the highest probability match and write down: the application deadline, what you still need to complete, who will own the submission process, and your internal deadline (at least 2 weeks before the actual deadline to allow for review).

Write your grant evidence map here:

---

---

---

---

---

## QUIZ

### 1. What type of capital are government grants?

A) Debt that requires repayment with interest B) Equity that dilutes your ownership C) Non-dilutive capital that does not require repayment D) Revenue-share arrangements with the government

### 2. What is the "key insight" of the Grant Evidence Engine's capture stage?

A) You need to create new evidence from scratch B) You're already doing the work, you're just not capturing it as evidence C) Grants don't actually require evidence D) Evidence must be notarized by a lawyer

### 3. Who sourced and itemized the grant templates referenced in this module?

A) Phillip B) Hitsuyo Aku C) Rob/Mahal D) Andre Houser El

### 4. What is the cited scale potential across qualifying grant programs?

A) 1 – 5M B) 5-10M C) 20 – 30M D) 100M+

### 5. What is described as "the only thing standing between you and this money"?

A) Better quality evidence B) A specialized lawyer C) Assigning someone to own the process D) More years of organizational history

## MODULE 8

# Cross-Cutting Patterns & Operational Playbook

This final module synthesizes the patterns that emerged across all six agents — the second-order insights that are more valuable than any individual system. These are the meta-principles that govern how to build, deploy, and scale agentic systems. Every pattern below was independently demonstrated by multiple people during a single session, which means they're not personality-dependent — they're structural laws of agentic operations. Internalize these patterns and you'll be able to design new agents for any business problem, not just the six covered in this workbook. These are the principles that separate someone who uses AI tools from someone who builds AI systems that run businesses.

### Token-Economy Discipline (The \$ Rule)

Both Hitsuyo (canary-test before batch) and Andre (spec-in-cheap-model, build-in-Fable) independently arrived at the same discipline: validate cheap, execute expensive. This is the fundamental law of AI production economics. Formalize it as a standing rule: "Spec in Sonnet/Cowork. Build in Fable. Never the reverse." Every dollar saved on exploration is a dollar available for production quality. This pattern was discovered independently by two different people working on two completely different problems, which is the strongest possible evidence that it's a structural law of agentic operations, not a personal preference or a niche trick. Apply this to every AI production workflow you build.

4 MOOLAH

## Single Point of Failure Elimination

Every agent in this workbook has a single point of failure. Cold Assassin equals Hitsuyo only. Meeting-to-Workflow equals Phillip only. Complaint Harvester equals Phillip only. This is the concentration-of-load risk — it's structural, not personal. The fix: every agent needs a documented SOUL.md (core operating principles), an SOP (step-by-step execution guide), and a backup operator who can run the system without the original owner. Without this documentation, every agent is one person's departure away from total failure. The SOUL.md captures the principles. The SOP captures the steps. The backup operator provides the insurance. All three are required.

DNA

## The Meta-Workflow First Principle

Agent 2 (Meeting-to-Workflow) should be automated as standing infrastructure before any other agent gets built. It's the system that generates all other systems. Every Git Built session, every pod call, every strategy sync should run through this by default. The standing automation: Zoom transcript export, webhook, Claude API, Notion staging, review. Build this first, and every subsequent agent gets documented for free. This is the highest-leverage first move because it accelerates the documentation of every system you build afterward. Think of it as buying a printing press before writing books — the press makes every book cheaper and faster to produce.

DNA

## Revenue Per Agent Math (The 4 MOOLAH Formula)

Each agent has a direct revenue model. Cold Assassin: 3-5 closes/month at 10K *equals* 30-50K. Complaint Harvester: 5 products at 47 *average*, 50 *sales each equals* 11,750/month. Game Builder: 1 game at 10K — 100K *on Steam*. Merch Drop : 200 *pre — orders at* 55 equals 11,000 *gross*. Grant Engine : 20-30M in potential awards. The formula: Output Volume times Conversion Rate times Average Revenue equals Monthly Agent Revenue. Calculate this for every agent you deploy. This formula turns vague enthusiasm into hard math, and hard math is what drives resource allocation decisions in a real business.

4 MOOLAH

## The Vendor Vetting Gate

During the session, unvetted vendor links (Etsy Disruptor, print partners) were dropped into Discord chat. Before routing real money through any vendor, a standard vetting pass is required: sample order, turnaround time test, quality check. This is the procurement discipline that separates professional operations from amateur mistakes. One bad vendor can destroy a product launch, damage your brand reputation with customers, and cost thousands in refunds and chargebacks. The vendor vetting gate is cheap insurance against expensive problems. Every vendor gets a sample order before any bulk commitment, period.

DNA

## The Concentration-of-Load Monitor

Track hours per team member per week. If any one person exceeds 60% of total operational load, the system is fragile. The fix: document, delegate, or automate the heaviest processes. The trigger: when one person's weekly hours exceed the threshold, immediately initiate handoff documentation for their top-loaded agent. This monitoring discipline ensures that growth doesn't create fragility. A system that depends on one person working 80-hour weeks isn't a system — it's a burnout waiting to happen. The 60% threshold is the early warning sign that prevents the catastrophic failure that comes when your single point of failure finally fails.

DNA

## \$ GIT MONEY

The 4 MOOLAH Formula is the master equation: Revenue equals Output Volume times Conversion Rate times Average Revenue. Apply it to every agent. Cold Assassin: 500 prospects/month times 2% reply rate times  $10K$  service equals 100K pipeline. Complaint Harvester: 5 products times 100 sales/month times  $47$  equals 23,500/month. Merch Drop: 1 line times 200 pre-orders times  $55$  equals 11,000/drop. Grant Engine: 10 applications times 20% win rate times  $50K$  average award equals 100K. Run the math on every process in your business. The agents that score highest on the 4 MOOLAH formula get your attention first. This is how you prioritize when everything seems important — you let the math decide.

## Calculate Your Agent Revenue Portfolio

1. **List every process in your business:** Write down 5-10 repeatable processes that currently require human effort. For each one, note: what it produces, who runs it, how many hours per week it takes, and what the output volume is (emails sent, products built, meetings held, etc.). Be thorough — include the invisible processes that "just happen" without anyone formally owning them.
2. **CLAW-map your top 3:** Take the three highest-effort processes from your list and write out the full CLAW structure for each one. Which stage is currently manual? Which stage could be automated today with existing tools? Which stage requires new tooling or integrations that don't exist yet?
3. **Calculate the 4 MOOLAH formula for each:** For each of your top 3 processes, estimate: monthly output volume, conversion rate (what percentage of outputs lead to revenue?), and average revenue per conversion. Multiply them together. Write the projected monthly revenue for each agent.
4. **Identify which agent has the highest revenue potential:** Compare your three 4 MOOLAH calculations side by side. Which process, if fully automated with a CLAW agent, would generate the most revenue per month? Write down the agent name and its projected monthly revenue number.
5. **Assign an owner to your highest-value agent:** Who will build this agent? Who will run it day-to-day? Who is the backup operator if the primary owner is unavailable? Write actual names, not just roles. If you can't name a backup operator, you've identified your single point of failure — fix it.

Write your revenue portfolio here:

---

---

---

---

---

## QUIZ

### 1. What is the formalized standing rule for token-economy discipline?

A) "Always use the cheapest available tools" B) "Spec in Sonnet/Cowork. Build in Fable. Never the reverse." C) "Spend more on premium models for better output" D) "Use the same model consistently for the entire project"

### 2. What three documents does every agent need to eliminate single points of failure?

A) Business plan, pitch deck, and financial projections B) SOUL.md, SOP, and backup operator assignment C) Mission statement, vision document, and values list D) API keys, prompt library, and output templates

### 3. Why should the Meeting-to-Workflow Agent be built before any other agent?

A) It's the easiest agent to build B) It generates the infrastructure that documents all other agents for free C) It generates the most direct revenue D) It was the first agent demonstrated in class

### 4. What is the 4 MOOLAH Formula?

A) Revenue = Profit Margin x Growth Rate x Market Size B) Revenue = Output Volume x Conversion Rate x Average Revenue C) Revenue = Leads Generated x Close Rate x Price Point D) Revenue = Website Traffic x CTR x Customer LTV

### 5. At what load percentage does the Concentration-of-Load Monitor trigger a handoff?

A) 30% of total operational load B) 40% of total operational load C) 60% of total operational load D) 80% of total operational load

## REFERENCE

# Cheat Sheet

Quick-reference table of all six agents covered in this workbook, organized by CLAW stage focus, revenue formula, tools, and current ownership.



| # | Agent Name              | CLAW Stage Focus                             | Tag | Revenue Formula                                      | Tools                                              | Owner           |
|---|-------------------------|----------------------------------------------|-----|------------------------------------------------------|----------------------------------------------------|-----------------|
| 1 | Cold Assassin           | All 4 stages + Self-Healing Loop             | DNA | 500 prospects/mo x 2% reply x 10K = 100K pipeline/mo | Public data scrape, Claude, email automation       | Hitsuyo Aku     |
| 2 | Meeting-to-Workflow     | C (Zoom AI) + L (CLAW auto-struct)           | DNA | Replaces \$150K/yr Chief of Staff                    | Zoom AI notes, Claude, Notion                      | Phillip         |
| 3 | Complaint Harvester     | C (multi-platform) + L (2-stage analysis)    | \$  | 5 products x 50 sales x 47 = 11,750/mo               | Reddit, Amazon, YouTube, Claude                    | Phillip         |
| 4 | Vibe-Coded Game Builder | C (concept brief) + A (spec-to-build)        | \$  | 1 game x \$10K-100K on Steam                         | Claude/Cowork, Fable 5, Godot, Discord, Higgsfield | Andre Houser El |
| 5 | Merch Drop Pipeline     | C (visual research) + A (tech pack)          | \$  | 200 pre-orders x 55 = 11,000/drop                    | IG, Google Images, Claude, factory partners        | Phillip / Andre |
| 6 | Grant Evidence Engine   | C (standing recording) + L (program mapping) | \$  | 10 apps x 20% win x 50K avg = 100K                   | Zoom, Claude, grant templates (Rob/Mahal)          | Unassigned      |

ASSESSMENT

# Final Assessment: 20 Questions

Test your understanding across all 8 modules. Circle or write your answers, then check against the answer sheet.

**1. What does CLAW stand for?**

A) Create, Launch, Analyze, Win B) Capture, Logic, Action, Workflow C) Configure, Learn, Automate, Repeat D) Code, Link, Adapt, Workflow

**2. In the CLAW framework, what is the key discipline of the Capture stage?**

A) Use the most expensive data source available B) The capture source must be automated or semi-automated, not manual C) Capture everything regardless of relevance D) Only capture data from paid premium sources

**3. What is the "Canary Test Protocol" used for?**

A) Testing AI model performance benchmarks B) Testing one output in isolation for AI slop before batch firing C) Measuring website traffic before a launch D) Validating data source reliability and accuracy

**4. What makes Cold Assassin's outreach "hyper-contextualized" rather than just "personalized"?**

A) It uses the prospect's first name in the subject line B) It diagnoses a real gap between claims and delivery using public data C) It mentions the prospect's competitors by name D) It's significantly longer than normal outreach emails

**5. How many parts are in the Locked Outreach Structure used by Cold Assassin?**

A) 3 parts B) 4 parts C) 5 parts D) 6 parts

**6. What is the single instruction that drives the Meeting-to-Workflow Agent's logic layer?**

A) "Summarize this meeting in bullet points" B) "Find action items and deadlines" C) "Create workflows out of our notes" D) "Write a project plan based on this transcript"

**7. Why is the Meeting-to-Workflow Agent considered the most important agent in the workbook?**

A) It generates the most direct revenue B) It's the easiest to build from scratch C) It's recursive — it generates the infrastructure that documents all other agents D) It was the first agent demonstrated during the live class

**8. What is the "Hidden Demand Gap" identified by the Complaint Harvester?**

A) Products that have zero reviews online B) The #1 desired outcome and the #1 obstacle blocking it C) Markets with no existing competitors D) Customer segments that no one is targeting

**9. How many product ideas does the Complaint Harvester generate per complaint pattern in Stage 2?**

A) 3 ideas B) 5 ideas C) 10 ideas D) 20 ideas

**10. What is the core token-economy principle demonstrated by the Vibe-Coded Game Builder?**

A) Always use the cheapest AI model for everything B) Spec in a cheap model, build in a premium model — never the reverse C) Use the premium model exclusively for all stages D) Avoid using AI for any creative work

**11. What is "Deliberate De-Derivatization" in the Game Builder pipeline?**

A) Removing all creative elements to avoid legal risk B) Actively preventing IP contamination by renaming references — borrow the genre, never the brand C) Making products as generic as possible to avoid lawsuits D) Using exclusively public domain assets for everything

**12. What is the Master Prompt Expansion formula in the Merch Drop Pipeline?**

A) Generate completely random designs for each product B) Lock the DNA with one master prompt, then systematically expand into 15 product variants C) Use the exact same prompt for every single product D) Let the AI autonomously design the full collection

**13. What is the locked production rule in the apparel pipeline?**

A) Always produce in bulk first to secure volume pricing B) Never use overseas manufacturing under any circumstances C) Strike-off before bulk, no exceptions D) Design must be 100% final before creating any mockup

**14. What type of capital are government grants?**

A) Debt capital requiring repayment with interest B) Equity capital that dilutes ownership stakes C) Non-dilutive capital that does not require repayment D) Revenue-share arrangements with government agencies

**15. What is the cited scale potential across all qualifying grant programs?**

A) 1 – 5million B) 5-10 million C) 20 – 30million D) 100 million or more

**16. What is the 4 MOOLAH Formula?**

A) Revenue = Profit Margin x Growth Rate x Market Size B) Revenue = Output Volume x Conversion Rate x Average Revenue C) Revenue = Leads Generated x Close Rate x Price Point D) Revenue = Website Traffic x CTR x Customer LTV

**17. What three documents does every agent need to eliminate single points of failure?**

A) Business plan, pitch deck, and financial projections B) SOUL.md, SOP, and backup operator assignment C) Mission statement, vision document, and values list D) API keys, prompt library, and output templates

**18. At what load percentage does the Concentration-of-Load Monitor trigger a handoff?**

A) 30% of total operational load B) 40% of total operational load C) 60% of total operational load D) 80% of total operational load

**19. What does the fact that two builders independently discovered the same token-economy discipline prove?**

A) AI models are getting cheaper over time B) The pattern is a structural law of agentic operations, not a personal preference C) You should only use one AI model per project D) Premium models are always superior to cheap models

**20. Which agent in this workbook is currently "unassigned" and explicitly needs an owner?**

A) Cold Assassin B) Meeting-to-Workflow Agent C) Complaint Harvester D) Grant Evidence Engine

# **BUILD THE MACHINES.**

Six agents. One framework. Infinite leverage. The operators who build these systems don't just work harder — they build the machines that work for them. That's the difference between earning money and printing it. GITBUILT.